

Multistage Graphs

A Comprehensive Technical Guide & Exam Revision Blueprint

1. Introduction

A Multistage Graph is a special type of Directed Acyclic Graph (DAG) that is divided into several stages. It is highly optimized and primarily utilized to solve complex sequential decision problems using the Dynamic Programming approach. It is explicitly used to find:

- Minimum cost path
- Maximum profit path
- Shortest route between a single source and a single destination

2. What is a Multistage Graph?

Definition

A Multistage Graph is a weighted directed graph whose vertices are divided into different disjoint sets or stages, and edges exist exclusively from vertices in one stage to vertices in the next consecutive stage. Dynamic Programming efficiently solves the graph by processing decisions backwards, moving from the destination stage towards the initial source stage.

3. Properties of Multistage Graph

1. **Directed Graph:** All edges have structural directionality.

$1 \rightarrow 2$

2. **Acyclic Graph:** Absolutely no feedback loops or cycles are present.

Correct (Acyclic):

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Wrong (Cyclic):

$1 \rightarrow 2 \rightarrow 3$ (with feedback cycle $3 \rightarrow 1$)

3. **Vertices Divided into Stages:** Vertices are structurally partitioned. For example:

Stage	Vertices
1	1
2	2, 3
3	4, 5, 6
4	7

4. **Single Source Vertex:** A unique starting node location where the path begins (e.g., **Source** = 1).
5. **Single Destination Vertex:** A unique final node location where the path terminates (e.g., **Destination** = 7).

4. Components of a Multistage Graph

- **Source Vertex:** The initial starting point of the structural pipeline (Node 1).
- **Destination Vertex:** The final ending point of the structural pipeline (Node 7).
- **Intermediate Vertices:** Disjoint processing nodes between the source and destination (Nodes 2, 3, 4, 5, 6).
- **Edge Weights:** Numerical representations of cost, distance, or resource expenditure.

1 ----5----> 2 (Cost = 5)

5. General Structure & Example Weighted Graph

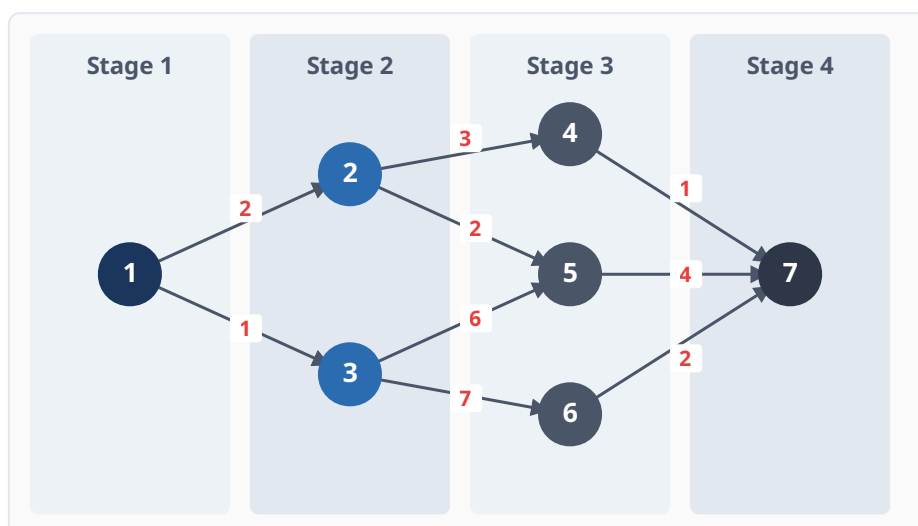


Figure 1: Fully connected Multistage Graph structure with Explicit Layer Boundaries and Edge Costs.

Edge Cost Table

Edge	Cost	Edge	Cost	Edge	Cost
1 → 2	2	2 → 4	3	4 → 7	1
1 → 3	1	2 → 5	2	5 → 7	4
		3 → 5	6	6 → 7	2
		3 → 6	7		

6. Dynamic Programming Principle

Dynamic Programming works fundamentally **backward**. Instead of blindly exploring every possible combination or path forward, we iteratively compute the minimum cost from each specific node to the final destination node.



Figure 2: Backward processing paradigm of Dynamic Programming.

7. Recurrence Relation

If $Cost(i)$ denotes the absolute minimum cost required to travel from node i to the destination, the recurrence formula is defined as:

$$Cost(i) = \min_{j \in Adj(i)} \{ c(i, j) + Cost(j) \}$$

Where:

- $c(i, j)$ = direct edge weight cost from node i to adjacent node j
- $Cost(j)$ = pre-computed minimum cost from node j to the destination node

8. Algorithm Step-by-Step

The processing sequence executes as follows:

- **Step 1:** Divide the graph configuration cleanly into disjoint sequential stages.
- **Step 2:** Initialize the absolute base destination cost: $Cost[destination] = 0$.
- **Step 3:** Start from the final stage and move recursively backward through preceding stages.

- **Step 4:** For each individual node, evaluate: $Cost(node) = \text{minimum}(\text{edge cost} + \text{child cost})$.
- **Step 5:** Cache and store the chosen best optimal child node to track the tracking map.
- **Step 6:** Reconstruct and trace the final operational path path from the source node to the destination.

9. Step-by-Step Solved Example Problem

Problem Statement:

Find the complete minimum cost path and optimal path tracing from node 1 to destination node 7 using the values from Figure 1.

Step 1: Partition Into Disjoint Stages

As initialized previously: Stage 1 = {1}; Stage 2 = {2, 3}; Stage 3 = {4, 5, 6}; Stage 4 = {7}.

Step 2: Initialize Destination Base Value

Since node 7 is the final terminal target: $Cost[7] = 0$.

Step 3: Solve Stage 3 (Moving Backward)

- **Vertex 4:** Transitions only to 7 (cost=1).
 $Cost(4) = 1 + 0 = 1$
- **Vertex 5:** Transitions only to 7 (cost=4).
 $Cost(5) = 4 + 0 = 4$
- **Vertex 6:** Transitions only to 7 (cost=2).
 $Cost(6) = 2 + 0 = 2$

Stage 3 DP Summary Table:

Vertex	Computed Minimum Cost
4	1
5	4
6	2
7	0

Step 4: Solve Stage 2

- **Vertex 2:** Evaluates two structural options:
 - Path through node 4: $3 + Cost(4) = 3 + 1 = 4$

- Path through node 5: $2 + \text{Cost}(5) = 2 + 4 = 6$
- Minimum Value Selection: $\text{Cost}(2) = \min(4, 6) = 4$. (Best child node = 4)
- **Vertex 3:** Evaluates two structural options:
 - Path through node 5: $6 + \text{Cost}(5) = 6 + 4 = 10$
 - Path through node 6: $7 + \text{Cost}(6) = 7 + 2 = 9$
 - Minimum Value Selection: $\text{Cost}(3) = \min(10, 9) = 9$. (Best child node = 6)

Step 5: Solve Source Vertex (Stage 1)

Node 1 evaluates two forward options:

- Through node 2: $2 + \text{Cost}(2) = 2 + 4 = 6$
- Through node 3: $1 + \text{Cost}(3) = 1 + 9 = 10$
- Minimum Value Selection: $\text{Cost}(1) = \min(6, 10) = 6$. (Best child node = 2)

Final Structured Dynamic Programming Table

Vertex	Minimum Cost	Next Vertex (Optimal Path Pointer)
1	6	2
2	4	4
3	9	6
4	1	7
5	4	7
6	2	7
7	0	-

Path Reconstruction Tracing

Reconstruction moves forward sequentially utilizing the tracked optimal markers:

1. Start at Source: **Node 1** → Next[1] = 2
2. Move to Node 2: **Node 2** → Next[2] = 4
3. Move to Node 4: **Node 4** → Next[4] = 7
4. Destination Reached.

Final Solved Result:

- Shortest Path: **1 → 2 → 4 → 7**
- Absolute Minimum Structural Cost: **6**

10. Pseudocode

```
// Multistage Graph Backward Induction Algorithm
cost[n] = 0; // Initialize destination cost

for(i = n-1; i >= 1; i--)
{
    cost[i] = INF;

    for(each vertex j adjacent to i)
    {
        if(cost[i] > edge(i,j) + cost[j])
        {
            cost[i] = edge(i,j) + cost[j];
            d[i] = j; // Cache the optimal next vertex tracking pointer
        }
    }
}
```

11. Complexity Analysis

- **Time Complexity:** $O(E)$ where E is the number of edges. This occurs because every structural link edge inside the stage architecture is analyzed exactly once.
- **Space Complexity:** $O(V)$ where V is the number of vertices. Storage tracking arrays are designated to contain the Cost values and Path pointers.

12. Advantages & Limitations

Advantages:

- **Highly Efficient:** Intelligently avoids redundant checking of all raw combinations.
- **Faster Execution:** Processes every edge structural line only once.
- **Optimal Framework:** Guarantees the verified minimum cost calculation path.

Limitations:

- **Restricted to DAGs:** Feedback loops or cyclic paths are completely prohibited.
- **Stage Structure Obligation:** Graph nodes must be partitioned into perfect stages.
- **Specialized Structure:** Ineffective for messy or arbitrary graph maps.

13. Applications

- **Resource Allocation:** Evaluating and assigning optimal minimum-cost items across stages.
- **Network Routing Frameworks:** Determining optimal path routing hops across routers.
- **Industrial Production Planning:** Managing assembly line processing stages sequentially.
- **Project Scheduling Tasks:** Mapping workflow deadlines to ensure minimized total processing cost.
- **Communication Networks:** Managing optimal data package streams over strict sequential gateways.

14. Core Exam Preparation Highlights

- Remember a Multistage Graph is structurally a **Directed Acyclic Graph (DAG)**.
- The initial source starts in Stage 1, while the target destination lives strictly in the final stage block.
- Dynamic Programming scans the options **backward** from target destination to source.
- The initialized baseline state is always: **$Cost[destination] = 0$** .
- Always maintain tracking arrays for both the **Minimum Cost** and the **Best Next Vertex Pointer** to enable reconstruction.

Last-Minute Quick Revision (60 Seconds Before Exam)

- **Core Definition:** A weighted directed acyclic graph divided entirely into clear structural stages. Solved using Dynamic Programming working from destination to source.
- **Core Formula:** $Cost(i) = \min \{ c(i, j) + Cost(j) \}$
- **Quick Path Steps:** $1 \rightarrow 2 \rightarrow 4 \rightarrow 7$ (Minimum Cost = 6).
- **Complexities:** Time = $O(E)$ | Space = $O(V)$

End of Technical Reference Note. Organized for Semester Performance.